

Transforming Code Patterns into Procedural Abstractions: An Empirical Study of De-com-po-si-tion

Georgiana Haldeman
Department of Computer Science
Colgate University
Hamilton, NY, United States
ghaldeman@colgate.edu

Claus Brabrand
Center for Comp. Ed. Research
IT University of Copenhagen
Copenhagen, Denmark
brabrand@itu.dk

Paul Denny
School of Computer Science
University of Auckland
Auckland, New Zealand
paul@cs.auckland.ac.nz

Abstract

Program decomposition is a core skill in computer science that overlaps with program comprehension, refactoring, and design. Decomposition takes many forms, with one common task at the introductory level being the identification and extraction of meaningful abstraction into separate functions. However, there is limited empirical evidence about how the algorithmic structuring of code affects the difficulty of producing a meaningful decomposition. In this paper, we empirically study how different algorithmic implementations of the same underlying task affect students' ability to reason about good abstractions through method extraction. Grounded in a recent framework on code structuring by Haldeman et al. [10], we design three functionally equivalent versions of the same task that differ only in how two latent functional patterns relate to each other: sequentially, hierarchically, or interleaved. Using a large-scale controlled study with 994 introductory programming students, we examine students' effort in decomposing the programs, the approaches they adopt, and their perceptions of whether the resulting decomposition improves readability and understanding. Our results show that interleaved functional composition is more difficult to decompose, providing initial empirical support for the framework's hypothesized ordering of composition pattern difficulty. Students also perceive their decompositions of the interleaved version to be less easy to read and understand. We also find that students describe many different approaches of decomposing, and we discuss several implications for teaching and future research.

CCS Concepts

• **Social and professional topics** → **Computing education**; • **Applied computing** → **Education**; • **Software and its engineering**

Keywords

Functional decomposition, code structuring, refactoring, method extraction, CS1, code patterns

ACM Reference Format:

Georgiana Haldeman, Claus Brabrand, and Paul Denny. 2026. Transforming Code Patterns into Procedural Abstractions: An Empirical Study of De-com-po-si-tion. In *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2026)*, July 10–15,

2026, Madrid, Spain. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3803400.3809385>

1 Introduction

Decomposition and abstraction are central skills in computer science and play a key role in program design and program comprehension [17, 31]. Being able to divide a larger problem into manageable pieces is essential for supporting higher-level reasoning about program behavior, and for producing readable and maintainable code [5, 15]. At the introductory level, the importance of good decomposition skills is amplified by generative AI [4]—as syntax becomes less of a bottleneck, students must increasingly engage with tasks such as evaluating and restructuring code, and decomposing problems into small pieces that can be addressed by an LLM [29].

One particularly important form of decomposition is bottom-up functional decomposition, which involves identifying and extracting coherent algorithmic patterns from existing code into functions to improve structure and readability. For novice programmers, such tasks can be challenging because they need to infer relationships from code that is unfamiliar to them [27]. Prior work has shown that carefully designed practice tasks can support students in developing code restructuring skills [12], but there is limited guidance for educators about appropriate task design and difficulty. Understanding the factors that determine the difficulty of code restructuring and decomposition tasks would help educators design and sequence activities that are both accessible and effective.

One recently proposed framework by [10] classifies code structuring tasks along three theoretically motivated dimensions: repetition, composition pattern, and data dependency [10]. The composition pattern dimension consists of three levels that describe how tasks are structurally combined within a program: sequentially (concatenation), nested (inclusion), or interwoven (interleaved). These levels are hypothesized to increase in complexity. Prior empirical work by Ginat et al. provides complementary evidence from a different setting, showing that when students are asked to construct programs, many novices avoid interleaved compositions even when they are optimal, and often misuse or malform them [9]. This suggests that interleaving poses conceptual challenges, but it remains unclear whether and how this is different when students are asked to decompose or restructure existing code rather than write it from scratch.

Moreover, student performance on such tasks is likely not determined by task complexity alone. Students' perceptions of the value of code decomposition likely also play an important role. Recent work suggests that students are more likely to engage productively with code refactoring when they recognise code quality issues and



This work is licensed under a Creative Commons Attribution 4.0 International License. *ITiCSE 2026, Madrid, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2634-7/2026/07

<https://doi.org/10.1145/3803400.3809385>

see refactoring as a way to improve attributes such as readability and structure [3]. This suggests that both the structural properties of the tasks and students' perceptions of their value may shape how such tasks are experienced and used in practice.

In this paper, we use the composition pattern dimension of [10]'s framework to design a set of decomposition tasks and we study how students engage with them. We compare three structurally distinct versions of the same underlying task: concatenated, inclusion, and interleaved, assigning one version to each of 994 introductory programming students. We investigate the following research questions, all implicitly centered around on the *differences* among the three different conditions:

- (RQ1) **Difficulty:** How difficult are the decomposition tasks in terms of how many students successfully complete them?
- (RQ2) **Effort:** How much effort does it take for students to decompose a program into meaningful parts?
- (RQ3) **Value:** How do students perceive the value of the decomposition task in terms of how much easier it is to read and understand the resulting code?
- (RQ4) **Approach:** What kinds of approaches do students describe using when attempting the code decomposition task?

2 Related Work

Decomposition and abstraction appear in many areas of computer science and computational thinking, such as code writing and code quality; it also requires program reading and comprehension abilities during code refactoring.

Code writing. Decomposition is at the core of code writing, as emphasized by Pattern-Oriented Instruction (POI) [22–24] and the theory of instruction for introductory programming skills [32]. POI is a pedagogical approach that refers to a methodology by which algorithmic patterns are incorporated into a course's instruction [23]. *Algorithmic patterns* represent good examples of elegant and efficient solutions to recurring algorithmic problems. Muller et al. [23] have shown that the incorporation of algorithmic patterns through POI enhances the construction of algorithmic problem-solving knowledge. Using a SOLO taxonomy for algorithmic design, Ginat and Menashe [8] also analyzed student solutions and challenges with problems at different levels of the taxonomy. Our work complements this prior work by studying different tasks.

Code quality. Decomposition is also discussed under the topic of code quality in software engineering. Several books on code quality, and software engineering more broadly, provide guidelines and principles for structuring code [7, 20, 21]. Java by Comparison [11] is a book on refactoring patterns similar to the book by Fowler [7] aimed at introductory courses. Novices may benefit from learning about code quality, including potentially increasing the efficiency of knowledge transfer and learning [13, 16]. Many novices and advanced students write code with poor style [13, 26]. Katona [16] found that students made fewer code comprehension mistakes when reading a properly styled code base and observed that *significantly less ... processing [was] necessary to understand the code*. However, Temporo et al. [28] found that the influence of function decomposition on code understanding is inconclusive, suggesting that functional decomposition does not universally enhance code comprehensibility.

Program comprehension. The Block Model [25], provides a framework for reasoning about a student's program reading and comprehension abilities. Izu et al. [14] applied the Block Model to learning activities targeted at the introductory level. Lehtinen et al. [18] used the Block Model to approximate the code comprehension abilities of LLMs.

In the present work, we map the themes identified in student free-text answers about their approach and challenges during the decomposition process onto the Block Model to further understand which approaches and abilities are needed to successfully decompose code. Our work also replicates and expands on some of the findings of Weeda et al. [30], in particular that *element interactivity impedes the ability to reason about code at an abstract level*.

3 Methodology

Our study addresses two *quantitative* research questions on **success** (RQ1) & **effort** (RQ2) as well as two *qualitative* research questions on **value** (RQ3) & **approach** (RQ4) as outlined in the introduction.

3.1 Participants

Data was collected from introductory programming at University of Auckland involving 994 students during the winter of 2025.

The course covers fairly traditional CS1 topics [19], but in the context of two programming languages: MATLAB and C. Our data was collected during the teaching of the C module, in the second half of the semester, shortly after the MATLAB module was delivered. Thus, at the time of our study, students had some prior exposure to programming principles (such as iteration and function calling) but in the context of the MATLAB language.

The module is structured around three in-person lectures each week, along with a complementary set of lab tasks. The lab tasks are automatically graded via a web-based tool, and are due at the beginning of the following week. Students can complete the lab tasks during an in-person lab session where teaching assistant help is available, or outside of class as homework. Each lab session contributes 1% towards the students' final grade. Our data was collected as part of the weekly lab session focusing on the topic of 'functions' (defining, calling, flow of control, and variable scope). This particular task contributed one tenth of the total 1% lab mark.

3.2 Design

We use a controlled between-subjects experimental design. Each student was randomly assigned one version of a function, asked to decompose it by defining at least one new function (i.e. a "helper" function), and then asked questions about their experience. The activity was preceded by the instruction: "*You will be shown a function and asked to decompose it into more than one function, without changing its behaviour. Sometimes, especially for complex tasks, this kind of refactoring can improve the readability of the code*". This study was approved by the UoA ethics committee.

3.3 Task & Treatment

Students were given a program to decompose into meaningful, coherent parts such that the resulting program exhibited the same behavior as defined by a test suite consisting of nine unit tests.

```

int Comp(int nums[], int Len) {
  int min = nums[0];
  int counter = 0;
  for (int i = 0; i < Len; i++) {
    int num = nums[i];
    if (min > num) {
      min = num;
    }
  }
  for (int j = 0; j < Len; j++) {
    int num = nums[j];
    if (num == min) {
      counter += 1;
    }
  }
  return counter;
}

```

(a) CONCATENATED decomposition (■ D1).

```

int Calc(int nums[], int Len) {
  int min = nums[0] + 1;
  int counter = 0;
  for (int i = 0; i < Len; i++) {
    int num = nums[i];
    if (min > num) {
      min = num;
      counter = 0;
      for (int j = 0; j < Len; j++) {
        if (nums[j] == min) {
          counter += 1;
        }
      }
    }
  }
  return counter;
}

```

(b) INCLUSION decomposition (■ D2).

```

int Proc(int nums[], int Len) {
  int min = nums[0];
  int counter = 0;
  for (int i = 0; i < Len; i++) {
    int num = nums[i];
    if (min > num) {
      min = num;
      counter = 1;
    } else if (min == num) {
      counter += 1;
    }
  }
  return counter;
}

```

(c) INTERLEAVED decomposition (■ D3).

Figure 1: The three different treatment programs of the study (D1–D3) each with a latent decomposition embedded within. (The dotted blue boxes towards the top and bottom are just common superficial programmatic idioms—*variable initialization* and *value return*—with specific placement; they contribute little towards the inherent complexity of a decomposition pattern).

The task used in this study involves counting the number of occurrences of the minimum value in a list. This task comprises two latent patterns—**finding the minimum** and **counting its occurrences**—which are composed in three distinct ways (see Figure 1): (a) sequential execution of the minimum pattern followed by the counting pattern (CONCATENATED), (b) execution of the counting pattern whenever a new minimum is discovered (INCLUSION), and (c) interleaved maintenance of the two patterns within a single traversal (INTERLEAVED).

Each student was randomly assigned to one treatment, each corresponding to one of these decomposition problems (see Figure 1):

- (D1) **CONCATENATED**, involving extraction of an initial for-loop sequentially followed by a subsequent for-loop into dedicated functions.
 - Requires: *identification of an appropriate cut point*.
- (D2) **INCLUSION**, involving extraction of an inner for-loop that is contextually surrounded by an outer for-loop into dedicated functions.
 - Requires: *multi-level abstraction (outer vs. inner focus)*.
- (D3) **INTERLEAVED**, involving extraction of one coherent code part that is semantically interwoven with another coherent code part into dedicated functions.
 - Requires: *relational disentanglement and reassembly*.

Task prompt. Along with the code corresponding to their treatment, students were given the instruction: “You should decompose the ‘Calc’ function above by defining at least one new function (i.e. a ‘helper’ function) that the ‘Calc’ function can then use. You can do this by moving some of the code currently in the ‘Calc’ function into a new function, and then calling this new function from the ‘Calc’ function. This process should simplify the definition of the ‘Calc’ function. Note: this decomposition must NOT change the functionality of the ‘Calc’ function (i.e. it should be functionally the same after the refactoring).”

After editing the code, students would use the ‘Check’ button to test the functionality of the code against the test suite. All failing

tests were shown to students, and they were permitted to modify and resubmit their code as often as they wished without penalty.

Post-task questions. After attempting the task, students were invited to respond to two post-task questions: one closed-form Likert item and one open-ended free-text prompt.

- (Q1) **Likert-scale.** “The refactored code (i.e., including the helper function) is easier to read and understand than the original code.”
- (Q2) **Free-text.** “In your own words, explain your approach to solving this refactoring task (i.e., why you decomposed the code the way you did, what you found challenging, etc.).”

3.4 Metrics

For each student, we collected the following background data:

- **ID**: an anonymous four-digit student ID ∈ [0000..9999];
- **Treatment** the student was randomly allocated to:

(D1) CONCATENATED, (D2) INCLUSION, or (D3) INTERLEAVED;

For each research question (RQ1–RQ4), we collected the following:

- (RQ– **Attempts**: For each attempted submission, we recorded:
 - 1) **Code** submitted as the decomposition attempt;
 - 2) **Timestamp** of the submission attempt; and
 - 2) **Status** of test suite on code: *success* (1) or *failure* (0);
- (RQ3) **Value** of the decomposition: To what extent the student considers the decomposed version *more-readable-than* the original (undecomposed) version, on a (1–5) Likert-scale:

Strongly agree; Agree; Neutral; Disagree; Strongly disagree.
- (RQ4) **Explanation**: Student free-text explanation (in their own words) of their approach to solving the decomposition task.

3.5 Decomposition Quality (of code produced)

All student code submissions that passed the autograder’s tests were manually analyzed by the first author, classified as either:

- **Failure:** No decomposition was performed, or decomposition was attempted but neither of the two targeted abstractions (*find minimum* and *count occurrences*) were identified and isolated; or
- **Success:** At least one of the two targeted abstractions was identified and hoisted into a dedicated function. This includes cases where one abstraction was extracted while the other remained embedded in the main function, as well as cases where the two abstractions were fully separated into independent functions.

This categorization is consistent with the prompt provided to the students (which asked them to decompose the code by defining at least one new function).

3.6 Thematic Analysis (of free-text answers)

The first author also thematically analyzed the open responses of the students describing their approach and the challenges during decomposition, using a process similar to Braun and Clarke’s [2] six-step framework. The first 100 answers were first read with the goal of familiarizing oneself with the data and generating the initial codes (steps 1 and 2 in Braun and Clarke [2]). The themes were then generated and then applied to the first 100 answers, while constantly being reviewed (steps 3 and 4 in Braun and Clarke [2]). Finally, the rest of the student answers were tagged with the refined themes.

Additionally, we also reflected on the implied difficulty of the themes using the Block Model of program comprehension [25] since decomposition involves many aspects of program comprehension. Although, the Block Model was initially proposed for instructors and researchers as a common language for discussing about program reading and comprehension [25], students’ depiction of their approach and challenges (1) still provides us some information of their reading and comprehension ability; and (2) could shed light on which comprehension abilities are needed to decompose specific code implementations when comparing the successful and unsuccessful students.

3.7 Data Analysis

As is customary, we adopt a 95% confidence interval ($\alpha = 5\%$) and symmetric two-tailed tests. We use Kruskal-Wallis to compare ordinal data for subjective usefulness of decompositions. We use the Z-test to compare ratios (e.g., $x/y \ll x'/y'$).

4 Results

4.1 Difficulty (RQ1)

Figure 2a shows the percentage of students successfully completing the task according to each of the three decomposition treatments (D1–D3). For the D1 (CONCATENATED) task, we see high success rates of 89% (278 out of 311). For D2 (INCLUSION), the success rate drops slightly, but significantly to 84% (269 out of 320) according to a Z-test ($p = 0.0434$). For D3 (INTERLEAVED), we see a much lower success rate of 55% (179 out of 328) which is strongly significant ($p < 0.00001$). We summarize:

FINDING 1 (Relative Decomposition Difficulty):

The difficulty of decompositions are strictly ordered:

(D1) CONCATENATED < (D2) INCLUSION $\ll$ (D3) INTERLEAVED.

where $x < y$ denotes y is-harder-than x ; & $x \ll y$ substantially so.

4.2 Effort (RQ2)

We investigate student effort using two behavioral proxies: number of submission attempts and struggle time (as introduced earlier).

Number of Attempts. Figure 2b shows the distribution of submission attempts across the three decomposition treatments (D1–D3). The distributions are highly overlapping, with similar proportions of students completing the task in few attempts and only a small minority requiring many submissions. Although a Kruskal–Wallis rank-based test detects a small distributional difference ($p = 0.0229$), the magnitude of this effect is minor and does not distinguish INCLUSION (D2) from INTERLEAVED decomposition (D3). (We see a similar pattern when restricting to successfully decompositions; cf. Section 4.1.)

Struggle Time. Figure 2c shows the corresponding distributions of inferred struggle time. As with submission attempts, the distributions are highly overlapping across treatments, with prolonged struggle confined to a small subset of students. A Kruskal–Wallis test reports that the three distributions are not statistically significantly different ($p = 0.2714$). In summary:

FINDING 2 (Comparable Effort):

Student effort is broadly comparable for CONCATENATED (D1), INCLUSION (D2), and INTERLEAVED (D3) decomposition.

Taken along with RQ1, this effectively means that effort (as measured in terms of number of submission attempts and struggle time) appears to be mostly independent of decomposition difficulty.

4.3 Value (RQ3)

Figure 3 shows students’ responses to whether the decomposed version of the program is *more-readable-than* the original, broken down by decomposition task (D1–D3). Throughout, perceived readability is used as a proxy for perceived value.

Pairwise χ^2 tests indicate no statistically significant difference in perceived value between CONCATENATED (D1) vs INCLUSION (D2): $p = 0.62$. In contrast, both CONCATENATED and INCLUSION are perceived as significantly more readable than INTERLEAVED decomposition (D3): $p = 0.013$, respectively, $p = 0.017$.

FINDING 3 (Perceived value of decomposition):

Overall, students found decomposition valuable: about 2/3 judged the decomposed version *more-readable-than* the original, around 1/4 were indifferent, and only a minority preferred the original code. This perceived value did not differ between CONCATENATED and INCLUSION decomposition, but was lower for INTERLEAVED decomposition (D3) with around 3/5 positive compared to about 2/3 for the other two (D1–D2).

4.4 Approach (RQ4)

Of the total 994 student, only 959 (97%) submitted code and answered the post-task questions. Of the 959 student responses manually analyzed (see Section 3.6), 76 (8%) contained no text and 67

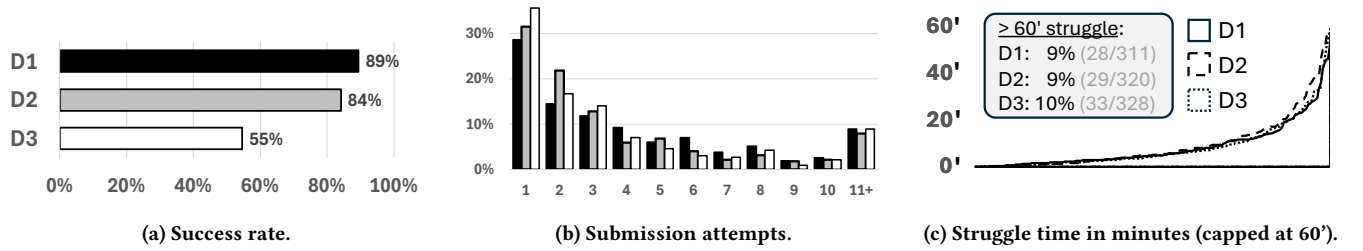


Figure 2: Difficulty (success rate) and Effort (submission attempts & struggle time) across decomposition treatments (D1–D3).

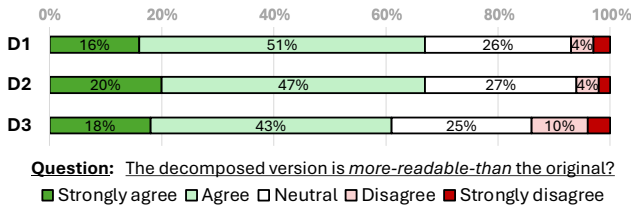


Figure 3: Preference for each decomposition task (D1–D3).

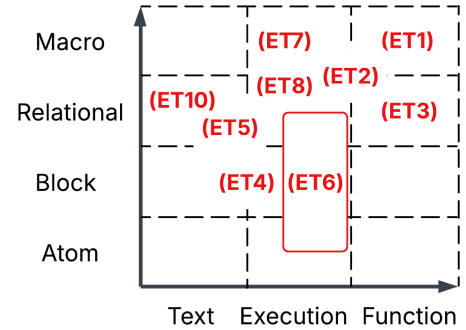


Figure 5: Mapping of the themes to the implied levels and dimensions of the Block Model of code comprehension [25].¹

(7%) contained unintelligible text or text that did not answer the question being asked. We identified the following themes in the rest of the 816 (85%) free-text responses:

- (ET1) **Find program purpose**; e.g., "I start by finding what the function does."
- (ET2) **Comprehension challenges**; e.g., "The challenging part was making sure I was understanding the code properly."
- (ET3) **Relate code components**; e.g., "I first analysed the steps that the function went through (finding the minimum value, counting the occurrences of that value)."
- (ET4) **Reference code blocks**; e.g., "[I] moved the first for loop into its own function."
- (ET5) **Handle syntax issues**; e.g., "The most challenging part was learning how to call one function from another."
- (ET6) **Handle data types**; e.g., "The most challenging part was figuring out the best way to implement pointers and make sure all the variables were changed properly."
- (ET7) **Preserve program behavior**; e.g., "The challenge was keeping the behavior exactly the same, especially the timing of when the count resets."
- (ET8) **Describe code behavior**; e.g., "The helper function finds the smallest number so that the main function only needs to count how many times it appears."
- (ET9) **Discuss code quality**; e.g., "[The] code is more readable and there is less to unpack."
- (ET10) **Algorithmic efficiency**; e.g., "[The code] was really inefficient and included nested loops which were unnecessary."

The 816 tagged student answers were then split based on the condition (D1, D2, or D3) and the quality of the decomposition (success vs. unsuccessful). Figure 4 shows the comparative distribution of the identified themes between successful and unsuccessful students (light blue vs. dark blue) separately for each of the three conditions. The trends corroborate our observations during the assessment of the decomposition quality of the code, namely:

- (F1) The majority of students relied on textual clues (expressed by referencing code blocks) when identifying the function to be extracted. However, this strategy only works for D1 and D2 code. For D3 code, the successful approach involves understanding the purpose of the code and how its different functional components relate to each other. Accordingly, we see the unsuccessful students talking less about the program purpose and relating code components.
- (F2) More students in D2 mentioned program comprehension and indicated that it was the most challenging for them probably because the code implementation involved nested loops, with several commenting on its algorithmic inefficiency.
- (F3) Reflecting on the results in Figure 4 with the mapping of the themes on the Block Model [25] in Figure 5¹, we conclude that students attempted the decomposition using primarily block level code comprehension skills (expressed by referencing code blocks). However, the successful decomposition required code comprehension skills at higher level such as relational and macro especially for D3 code.

¹Note that (ET9), discussing code quality, is not represented because it does not specifically imply a program comprehension activity.

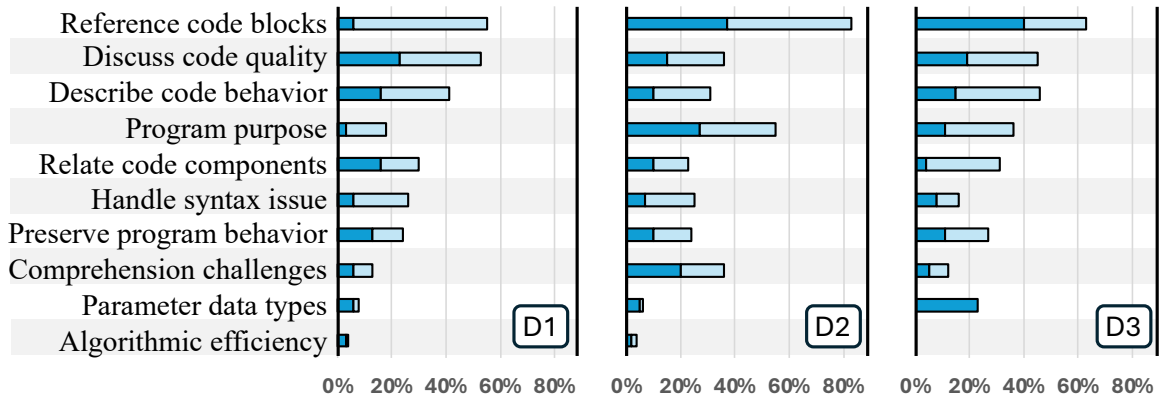


Figure 4: Distribution of themes for each treatment (D1–D3) along with failed (dark blue) vs. successful (light blue) decomposition.

FINDING 4 (Approach): The majority of students relied heavily on textual clues when decomposing. However, the successful students mentioned understanding the program’s function and the relation between its parts more than the unsuccessful students.

5 Threats to Validity

Task design and external validity. From an algorithmic perspective, the task used in this study was deliberately simple which may have limited the observed performance differences between groups. However, by holding algorithmic complexity constant and minimal, we are able to isolate the effect of decomposition structure which is the main goal of this work.

In addition, our use of a single, tightly controlled task may appear to limit generalizability. Flyvbjerg identifies as a common misunderstanding the claim that:

MISUNDERSTANDING 1: “One cannot generalize from a single case, therefore, the single-case study cannot contribute to scientific development.” [6]

This has been debunked as an invalid (false) scientific claim [6]. Rather than aiming for breadth across tasks, our goal was to treat the task as an *existence proof* that compositional structure alone can measurably affect the recovery of a latent decomposition. Future work should explore a wider range of tasks to examine how robust the effect is across different programming languages, contexts and patterns of composition.

Interpretation of student behaviour. There is a risk that some students treated the task as a mechanical refactoring exercise (“extract method”) rather than as a conceptual decomposition task. If students focused mainly on surface-level code changes rather than understanding the underlying structure, our measures of effort and perception may underestimate the true difficulty of decomposition, especially for interleaved code. We mitigated this threat by triangulating quantitative performance data with students’ qualitative explanations of their approach.

There is also a risk that students were not directly monitored while completing the task and may have received external help (e.g.,

from peers or other resources). However, this reflects the authentic conditions under which students normally work on lab tasks in the course. Moreover, because students were randomly assigned to treatments, there is no reason to believe that any external assistance would advantage one condition over another. If anything, external help would tend to reduce differences between groups. Finally, the task contributed only 0.1% towards the students’ final grade, which is a minimal incentive for seeking external help.

Measurement of effort. Task duration (which was just one measure related to RQ2) was not measured directly, but inferred from timestamped submission attempts. Since there is no explicit start-time marker (i.e., “timestamp zero”), students who solved the task on their first attempt cannot be assigned a task duration, and for students with multiple attempts only subsequent intervals (i.e., $n-1$ timestamps) are observable. Accordingly, inferred duration is best interpreted as a measure of *struggle duration* rather than absolute time-on-task, and is used cautiously to compare relative trends across treatments.

6 Conclusion

Program decomposition is a foundational skill for code comprehension and quality, yet little empirical work has isolated how compositional structure alone shapes its difficulty. Using three functionally equivalent but structurally different implementations of the same task, we showed that interleaved composition is substantially harder for novices to decompose than concatenated or inclusion structures, even when student effort is comparable across conditions. Our results provide initial empirical support for theoretically hypothesised orderings of composition pattern difficulty. These findings can inform the design and sequencing of decomposition tasks that scaffold students from simpler to more complex compositional structures. Furthermore, the difficulty of the three *extract method* tasks also correlates with the success rate of automating their corresponding refactoring [1]. Thus, our results suggest that our students are not prepared to overcome the limitations of existing automated refactoring tools or those built on top of LLMs [33].

References

- [1] Alireza Ardalani, Saeed Parsa, Morteza Zakeri-Nasrabadi, and Alexander Chatzigeorgiou. 2023. Supporting Single Responsibility Through Automated Extract Method Refactoring. *Empirical Software Engineering* 29, 28 (2023).
- [2] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative research in psychology* 3, 2 (2006), 77–101.
- [3] Eduardo Carneiro Oliveira, Hieke Keuning, and Johan Jeuring. 2024. Investigating Student Reasoning in Method-Level Code Refactoring: A Think-Aloud Study. In *Proceedings of the 24th Koli Calling International Conference on Computing Education Research (Koli Calling '24)*. Association for Computing Machinery, New York, NY, USA, Article 8, 11 pages. doi:10.1145/3699538.3699550
- [4] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing Education in the Era of Generative AI. *Commun. ACM* 67, 2 (Jan. 2024), 56–67. doi:10.1145/3624720
- [5] Kezia Devathasan, Celina Berg, and Daniela Damian. 2022. The Role of Abstraction in Introductory Programming. In *Proceedings of the 2022 ACM SIGPLAN International Symposium on SPLASH-E (Auckland, New Zealand) (SPLASH-E 2022)*. Association for Computing Machinery, New York, NY, USA, 7–13. doi:10.1145/3563767.3568125
- [6] Bent Flyvbjerg. 2006. Five Misunderstandings About Case-Study Research. *Qualitative Inquiry* 12, 2 (2006), 219–245. doi:10.1177/1077800405284363
- [7] Martin Fowler. 2018. *Refactoring: improving the design of existing code*. Addison-Wesley Professional.
- [8] David Ginat and Eti Menashe. 2015. SOLO Taxonomy for assessing novices' algorithmic design. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. 452–457.
- [9] David Ginat, Eti Menashe, and Amal Taya. 2013. Novice difficulties with interleaved pattern composition. In *Informatics in Schools. Sustainable Informatics Education for Pupils of all Ages: 6th International Conference on Informatics in Schools: Situation, Evolution, and Perspectives, ISSEP 2013, Oldenburg, Germany, February 26–March 2, 2013. Proceedings* 6. Springer, 57–67.
- [10] Georgiana Haldeman, Peter Ohmann, and Paul Denny. 2026. Systematically Thinking about the Complexity of Code Structuring Exercises at Introductory Level. In *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V. 1*. 442–448.
- [11] Simon Harter, Linus Dietz, and Jörg Lenhard. 2018. Java by Comparison: Become a Java Craftsman in 70 Examples. (2018).
- [12] Cruz Izu, Paul Denny, and Sayoni Roy. 2022. A Resource to Support Novices Refactoring Conditional Statements. In *Proceedings of the 27th ACM Conference on Innovation and Technology in Computer Science Education Vol. 1 (Dublin, Ireland) (ITICSE '22)*. Association for Computing Machinery, New York, NY, USA, 344–350. doi:10.1145/3502718.3524810
- [13] Cruz Izu, Claudio Mirolo, Jürgen Börstler, Harold Connamacher, Ryan Crosby, Richard Glassey, Georgiana Haldeman, Olli Kiljunen, Amruth N Kumar, David Liu, et al. 2025. Introducing Code Quality at CS1 Level: Examples and Activities. In *2024 Working Group Reports on Innovation and Technology in Computer Science Education*. 339–377.
- [14] Cruz Izu, Carsten Schulte, Ashish Aggarwal, Quintin Cutts, Rodrigo Duran, Mirela Gutica, Birte Heinemann, Eileen Kraemer, Violetta Lonati, Claudio Mirolo, et al. 2019. Fostering program comprehension in novice programmers-learning activities and learning trajectories. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education*. 27–52.
- [15] Maria Kallia and Sue Sentance. 2021. Threshold concepts, conceptions and skills: Teachers' experiences with students' engagement in functions. *Journal of Computer Assisted Learning* 37, 2 (2021), 411–428.
- [16] Jozsef Katona. 2021. Clean and Dirty Code Comprehension by Eye-tracking Based Evaluation using GP3 Eye Tracker. 18, 1 (2021), 79–99. doi:10.12700/APH.18.1.2021.1.6
- [17] Jeff Kramer. 2007. Is abstraction the key to computing? *Commun. ACM* 50, 4 (April 2007), 36–42. doi:10.1145/1232743.1232745
- [18] Teemu Lehtinen, Charles Koutchme, and Arto Hellas. 2024. Let's ask ai about their programs: Exploring chatgpt's answers to program comprehension questions. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training*. 221–232.
- [19] Andrew Luxton-Reilly, Simon, Ibrahim Albluwi, Brett A. Becker, Michail Giannakos, Amruth N. Kumar, Linda Ott, James Paterson, Michael James Scott, Judy Sheard, and Claudia Szabo. 2018. Introductory programming: a systematic literature review. In *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (Larnaca Cyprus, 2018-07-02)*. ACM, 55–106. doi:10.1145/3293881.3295779
- [20] Robert C Martin. 2009. *Clean code: a handbook of agile software craftsmanship*. Pearson Education.
- [21] Steve McConnell. 2004. *Code complete*. Pearson Education.
- [22] Orna Muller. 2005. Pattern oriented instruction and the enhancement of analytical reasoning. In *Proceedings of the 2005 international workshop on Computing education research - ICER '05 (Seattle, WA, USA, 2005)*. ACM Press, 57–67. doi:10.1145/1089786.1089792
- [23] Orna Muller, David Ginat, and Bruria Haberman. 2007. Pattern-oriented instruction and its influence on problem decomposition and solution construction. In *Proceedings of the 12th annual SIGCSE conference on Innovation and technology in computer science education*. 151–155.
- [24] Orna Muller, Bruria Haberman, and Haim Averbuch. 2004. (An almost) pedagogical pattern for pattern-based problem-solving instruction. (2004).
- [25] Carsten Schulte. 2008. Block Model: an educational model of program comprehension as a tool for a scholarly approach to teaching. In *Proceedings of the Fourth international Workshop on Computing Education Research (Sydney Australia, 2008-09-06)*. ACM, 149–160. doi:10.1145/1404520.1404535
- [26] Anshul Shah, Thomas Rexin, Gonzalo Allen-Perez, Kevin Wu, William G Griswold, and Adalbert Gerald Soosai Raj. 2025. Identifying Students' Code Quality Defects while Contributing to Large Code Bases. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1*. 514–520.
- [27] Ivan Tan and Christopher M. Poskitt. 2024. Fixing Your Own Smells: Adding a Mistake-Based Familiarisation Step When Teaching Code Refactoring. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (Portland, OR, USA) (SIGCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 1307–1313. doi:10.1145/3626252.3630856
- [28] Ewan Tempero, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, Diana Kirk, Juho Leinonen, Asma Shakil, Robert Sheehan, James Tizard, Yu-Cheng Tu, et al. 2024. On the comprehensibility of functional decomposition: An empirical study. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*. 214–224.
- [29] Annapurna Vadaparty, Daniel Zingaro, David H. Smith Iv, Mounika Padala, Christine Alvarado, Jamie Gorson Benario, and Leo Porter. 2024. CS1-LLM: Integrating LLMs into CS1 Instruction. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1 (Milan Italy, 2024-07-03)*. ACM, 297–303. doi:10.1145/3649217.3653584
- [30] Renske Weeda, Sjaak Smetsers, and Erik Barendsen. 2024. Unraveling novices' code composition difficulties. *Computer Science Education* 34, 3 (2024), 414–441.
- [31] Jeannette M. Wing. 2006. Computational thinking. *Commun. ACM* 49, 3 (March 2006), 33–35. doi:10.1145/1118178.1118215
- [32] Benjamin Xie, Dastyni Loksa, Greg L. Nelson, Matthew J. Davidson, Dongsheng Dong, Harrison Kwik, Alex Hui Tan, Leanne Hwa, Min Li, and Amy J. Ko. 2019. A theory of instruction for introductory programming skills. 29, 2 (2019), 205–253. doi:10.1080/08993408.2019.1565235
- [33] Yisen Xu, Feng Lin, Jinqiu Yang, Nikolaos Tsantalas, et al. 2025. Mantra: Enhancing automated method-level refactoring with contextual RAG and multi-agent LLM collaboration. *arXiv preprint arXiv:2503.14340* (2025).